

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2-DEBUGGING & OPTIMISATION TASK

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL3DEBUGGING & OPTIMISATION TASK
Weightage:	20%	Total Marks:	25
CO2 Statement:	<i>Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)</i>		
Student Name:	P.LOGESH	Reg.No.:	192521351
Department:	B.TECH IT	Date:	31/07/2026

ANNEXURE A

1. A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.

1. Identification of Errors / Bugs

- Inputs:
 - +120 = 01111000
 - +50 = 00110010
- Binary Addition:
01111000
+ 00110010

10101010
- Result = 10101010 = -86 (incorrect)
- Expected Result = 170
- Since 170 cannot be represented in 8-bit signed format (-128 to +127), overflow occurs.

2. Debugging Approach & Analysis

Check overflow condition.

Rule:

- Adding two positive numbers gives a negative result.

- OR adding two negative numbers gives a positive result.

Here,

Positive + Positive = Negative

Hence,

Overflow Detected = YES

3. Correctness of Debugged Solution

Overflow detection logic:

Overflow =

$(A7 == B7) \text{ AND } (\text{Result7} \neq A7)$

Where A7, B7 and Result7 are the sign bits.

For this example,

A7 = 0

B7 = 0

Result7 = 1

Overflow = TRUE

The processor correctly identifies overflow instead of accepting the wrong answer.

4. Optimization Techniques Applied

- Add overflow detection circuitry.
- Generate overflow flag.
- Use wider data type (16-bit or 32-bit).
- Display overflow exception.

5. Testing and Documentation

Test Case	Expected	Result
120 + 50	Overflow	Pass
20 + 30	50	Pass
-70 + -80	Overflow	Pass
40 + -20	20	Pass

2. A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry lookahead adder, and explain how it reduces delay.

1. Identification of Errors / Bugs

The ripple carry adder is functionally correct but suffers from high propagation delay because each carry must travel through every full adder.

Example:

Bit0 $\rightarrow$ Bit1 $\rightarrow$ Bit2 $\rightarrow$... $\rightarrow$ Bit7

Each stage waits for the previous carry.

2. Debugging Approach & Analysis

Delay increases with the number of bits.

Example:

C0

↓

FA0

↓

C1

↓

FA1

↓

C2

↓

...

C8

Total delay

Delay = $n \times$ Full Adder Delay

3. Correctness of Debugged Solution

Replace Ripple Carry Adder with Carry Lookahead Adder.

Carry equations:

$$P_i = A_i \text{ XOR } B_i$$

$$G_i = A_i \text{ AND } B_i$$

$$C_{i+1} = G_i + P_i C_i$$

Carries are generated simultaneously.

4. Optimization Techniques Applied

- Carry Lookahead Adder
- Parallel carry generation
- Reduced propagation delay
- Faster arithmetic operations

5. Testing and Documentation

Adder	Delay	Performance
Ripple Carry	High	Slow
Carry Lookahead	Low	Fast

Performance improved significantly.

- 3. A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bitpair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.**

1. Identification of Errors / Bugs

Incorrect multiplication occurs because of:

- Wrong bit-pair checking
- Incorrect arithmetic right shift
- Improper sign extension

Negative operands produce incorrect results.

2. Debugging Approach & Analysis

Verify each iteration.

Check:

Q0 Q-1

Rules

01 → Add M

10 → Subtract M

00 → No operation

11 → No operation

After operation,

Perform

Arithmetic Right Shift

Ensure sign bit remains unchanged.

3. Correctness of Debugged Solution

Corrections:

- ✓ Correct bit-pair detection
- ✓ Preserve sign bit

✓ Correct arithmetic shift

✓ Proper sign extension

Now negative multiplication produces correct answer.

4. Optimization Techniques Applied

- Correct control logic
- Automatic sign extension
- Validate each iteration
- Error checking after every shift

5. Testing and Documentation

Operands	Expected	Result
5×3	15	Pass
-5×3	-15	Pass
-5×-3	15	Pass
7×-4	-28	Pass

4. An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.

1. Identification of Errors / Bugs

The algorithm restores the previous remainder whenever subtraction becomes negative.

This causes:

- Extra additions
- Increased execution time

2. Debugging Approach & Analysis

Workflow:

Subtract

↓

Negative?

↓

Yes

↓

Restore

↓

Repeat

Repeated restoration is inefficient.

3. Correctness of Debugged Solution

Replace Restoring Division with Non-Restoring Division.

Instead of restoring:

Negative

↓

Next iteration performs addition

No immediate restoration.

Produces the same quotient with fewer operations.

4. Optimization Techniques Applied

- Non-Restoring Division
- Eliminate unnecessary restoration
- Reduce arithmetic operations
- Improve execution speed

5. Testing and Documentation

Method	Operations	Speed
Restoring	High	Slow
Non-Restoring	Low	Fast

Execution time reduced.

- 5. A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.**

1. Identification of Errors / Bugs

Example:

1×10^{30}

+

1

The small value disappears after exponent alignment.

Causes:

- Loss of precision
- Rounding error
- Normalization issues

2. Debugging Approach & Analysis

Floating-point addition process:

Align Exponents

↓

Shift Mantissa

↓

Add Mantissas

↓

Normalize

↓

Round

Large exponent difference causes the smaller mantissa to be shifted out.

3. Correctness of Debugged Solution

Improve accuracy by:

- IEEE 754 Double Precision (64-bit)
 - Kahan Summation Algorithm
 - Higher precision arithmetic
- Results become significantly more accurate.

4. Optimization Techniques Applied

- Double precision
- Kahan summation
- Pairwise summation
- Improved rounding methods

5. Testing and Documentation

Precision	Result	Accuracy
Single Precision	Precision Loss	Low
Double Precision	Accurate	High

The optimized approach minimizes rounding errors and improves numerical stability.

Code of Conduct:

I P.LOGESH(192521351) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: P.LOGESH

MARKS ALLOCATION

	Identification of Errors / Bugs(20%)	Debugging Approach & Analysis(20%)	Correctness of Debugged Solution(25%)	Optimization Techniques Applied(20%)	Testing and Documentation(15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Identification of Errors / Bugs	20%	Accurately identifies all errors and issues with complete understanding of the problem	Identifies most errors with minor omissions	Identifies limited errors; misses key issues	Unable to identify errors or misunderstands the problem
Debugging Approach & Analysis	20%	Uses effective debugging techniques with clear analysis and root cause identification	Applies suitable debugging methods with minor gaps	Uses basic debugging methods with limited analysis	No systematic debugging approach followed
Correctness of Debugged Solution	25%	Provides completely corrected solution with accurate functionality and expected output	Provides working solution with minor errors	Partially correct solution with limited functionality	Incorrect solution or fails to resolve errors
Optimization Techniques Applied	20%	Applies efficient optimization methods to improve performance, memory usage and quality	Performs optimization with noticeable improvements	Limited optimization with minimal improvements	No optimization applied or inefficient implementation
Testing and Documentation	15%	Performs complete testing with proper validation and clear documentation	Provides adequate testing and documentation with minor issues	Limited testing and incomplete documentation	No proper testing or documentation provided

